

WeFlowBridge

给我（主脑）、Cline 以及任何接入的 AI 看的一份说明：**本机的 WeFlow 微信 API 能调什么、有什么用、怎么安全地用。**

这是一个**WeFlow 数据源适配器项目**：公开仓库只保存 API 行为说明、自检/看门狗脚本和 AI 消费契约，**不保存任何密钥或原始微信数据**。真实 token、数据库密钥等只存在本地 `.env`（已被 `.gitignore` 忽略）。

✦ **适用版本**：WeFlow **26.5.27**（`C:\Program Files\WeFlow\WeFlow.exe`）· **实测日期**：2026-06-20 WeFlow 持续更新，接口可能随版本变动。本文端点 = 源码（`app.asar`）抽取 + 本机实测双重确认；换版本后请重跑 `probe-weflow.ps1` 复核，并更新此处版本/日期。

1. 这是什么

WeFlow 是一个**完全本地**运行的微信（4.0+）聊天记录查看 / 分析 / 导出工具。它在本机起一个 HTTP 服务，把"读取并解密本地微信消息"的能力映射成 REST 接口，供外部脚本、自动化、以及 **AI Agent** 调用。

- **服务地址**：`http://127.0.0.1:5031`（仅监听本地回环，不对外网开放）
- **启用方式**：WeFlow 客户端 → 设置 → API 服务 → 启动服务
- **输出格式**：原始 JSON，或 ChatLab 标准格式
- **状态**：官方标注"早期阶段，接口可能变动"——以本机实测为准（见 `probe-weflow.ps1`）

⚠ 解密所需的数据库密钥 / 图片密钥都由 **WeFlow 内部持有并使用**，**不是 API 入参**。调接口只需要 base URL（+ token）。

项目边界

- 本仓库是 `E:\WeFlowBridge`，负责 WeFlow API 适配、公开安全文档、自检和看门狗。
- AI 调用层在 `E:\.agents\plugins\weflow-toolkit`，不在本仓库保存长期个人事实。
- `PersonalOS`、`CareerCapital`、`SocialCapital`、`LifeCases` 等下游项目只应按需引用 WeFlow 证据，不吞完整微信库。
- AI 消费契约见 `docs/ai_consumer_contract.md`。
- 公开仓库隐私边界见 `docs/privacy_boundary.md`。

2. 它能用来干嘛（对 AI 而言）

场景	怎么实现
检索某人/某群的聊天记录	<code>GET /api/v1/messages?talker=<id></code> ，支持时间范围、关键词、分页
让 AI 总结/分析一段对话	拉 <code>chatlab=1</code> 标准格式 → 直接喂给模型做摘要、情绪、画像
列出所有会话 / 找某个会话	<code>GET /api/v1/sessions?keyword=工作群</code>
导出联系人	<code>GET /api/v1/contacts?keyword=张三</code>
群成员画像（谁是群主/发言量）	<code>GET /api/v1/group-members?talker=<群id></code> ，含 <code>isOwner</code> / <code>messageCount</code> 等
读/分析朋友圈	<code>GET /api/v1/sns/timeline</code> （含点赞、评论、定位）+ <code>GET /api/v1/sns/export/stats</code>

场景	怎么实现
拿到图片/语音/视频文件	查询时 <code>media=1</code> 导出，再用 <code>/api/v1/media/...</code> 取文件
健康检查 / 服务是否在线	<code>GET /health</code>

一句话：它把“我的微信历史 + 朋友圈”变成一个 AI 可读的本地数据源——可以做检索、摘要、年度报告、双人关系分析、群画像、朋友圈分析、媒体提取等。

完整端点（含上面没列全的朋友圈/防删等）见下方 §4.5 完整端点地图。

3. 接口参考

本节列出最常用的只读查询接口（均为 `GET`），`base = http://127.0.0.1:5031`。完整端点（含 `POST` / `DELETE` 写操作、朋友圈、推送）见下方 §4.5 完整端点地图。

`GET /health` — 健康检查

返回 `{ "status": "ok" }`。用来判断服务是否启动。

`GET /api/v1/sessions` — 会话列表

参数	必填	说明
<code>keyword</code>	✗	按会话名 / ID 搜索
<code>limit</code>	✗	数量上限，默认 100

返回 `sessions[]`（2026-06-20 实测字段，与官方文档略有出入）：`username` / `displayName` / `type` / `sessionType`（`private` | `group`） / `lastTimestamp`（秒级时间戳） / `unreadCount`。

`GET /api/v1/contacts` — 联系人列表

参数	必填	说明
<code>keyword</code>	✗	搜索关键词
<code>limit</code>	✗	数量上限，默认 100

返回 `contacts[]`（2026-06-20 实测字段）：`username` / `displayName` / `nickname` / `type`。

`GET /api/v1/messages` — 消息（核心接口）

参数	必填	说明
<code>talker</code>	✓	会话 ID (wxid 或群 ID)
<code>limit</code>	✗	返回条数，默认 100，范围 1~10000
<code>offset</code>	✗	分页偏移，默认 0
<code>start / end</code>	✗	时间范围，格式 <code>YYYYMMDD</code>
<code>keyword</code>	✗	按消息显示文本过滤
<code>chatlab</code>	✗	<code>1</code> = 输出 ChatLab 格式
<code>format</code>	✗	<code>json</code> （默认）或 <code>chatlab</code>

参数	必填	说明
media	✗	1 = 同时导出媒体并返回路径； 0 = 占位符
image / voice / video / emoji	✗	media=1 时分别控制各类媒体导出（ 1/0 ）

媒体默认导出到（实测） `%APPDATA%\weflow\cache\api-media`。

⚠ **实测必读：** `/messages` 有两种稳定用法。取**最新消息**时不要依赖日期窗口，直接 `GET /api/v1/messages?talker=<id>&limit=100`，把返回数组按 `createTime` 降序理解，最新在索引 0，并用会话 `lastTimestamp` 与第一条消息 `createTime` 对齐自检。取**历史区间/批量回溯**时再显式给 `start / end`（如 `start=20250101&end=20261231`）。返回 0 可能是实时读库竞态，需重试，不要直接判断为无消息。

示例

```
GET /api/v1/messages?talker=wxid_xxx&limit=50
GET /api/v1/messages?talker=wxid_xxx&chatlab=1
GET /api/v1/messages?talker=wxid_xxx&start=20260101&end=20260205&limit=100
GET /api/v1/messages?talker=wxid_xxx&keyword=项目进度&limit=50
GET /api/v1/messages?talker=wxid_xxx&media=1&image=1&voice=1&video=0&emoji=0
```

`GET /api/v1/media/{relativePath}` — 取已导出的媒体

例： `/api/v1/media/wxid_xxx/images/image_123.jpg`。需先用 `media=1` 导出才能访问。支持 `png/jpg/gif/webp/wav/mp3/mp4`。

ChatLab 消息类型映射

0 文本 · 1 图片 · 2 语音 · 3 视频 · 4 文件 · 5 表情 · 7 链接 · 8 位置 · 20 红包 · 21 转账 · 23 通话 · 80 系统 · 81 撤回 · 99 其他

4. 给 AI / 主脑 / Cline 的接入说明

绝不要把 token / 密钥 / wxid 写进代码或仓库。一律从本地 `.env` 读：

AI 消费者必须遵守 `docs/ai_consumer_contract.md`：先判断当前 WeFlow 库，最新消息不带 `start/end`，输出或内部保留 `current_library`、`target_conversation`、`talker`、`time_window`、`retry_count`、`message_count`、`lastTimestamp_matches_newest` 等字段。隐私与公开提交红线见 `docs/privacy_boundary.md`。

```
# PowerShell: 从 .env 读配置后调用
$cfg = @{}; Get-Content "E:\WeFlowBridge\.env" | Where-Object { $_ -match '^s*[^\#].*=' } | ForEach-Object {
    $k,$v = $_ -split '=',2; $cfg[$k.Trim()] = $v.Trim() }
$base = $cfg['WEFLOW_BASE_URL']
$headers = @{}; if ($cfg['WEFLOW_TOKEN']) { $headers['Authorization'] = "Bearer $($cfg['WEFLOW_TOKEN'])" }

Invoke-RestMethod "$base/health" -Headers $headers
Invoke-RestMethod "$base/api/v1/sessions?limit=5" -Headers $headers
Invoke-RestMethod "$base/api/v1/messages?talker=wxid_xxx&chatlab=1&limit=20" -Headers $headers
```

```
# Python
import os, requests
base = os.environ["WEFLOW_BASE_URL"]
h = {"Authorization": f"Bearer {os.environ['WEFLOW_TOKEN']}" if os.environ.get("WEFLOW_TOKEN") else {}}
msgs = requests.get(f"{base}/api/v1/messages",
                    params={"talker": "wxid_xxx", "chatlab": 1, "limit": 100}, headers=h).json()
```

实测 (2026-06-20)：本机这版 WeFlow 对数据接口强制鉴权——不带 token 返回 401。/health 例外 (无需 token)。缺 talker 返回 400。

token 三种写法都接受 (源码 verifyToken 实证)：① 请求头 Authorization: Bearer <token>；② query ? access_token=<token>；③ body 里 access_token。浏览器 EventSource (SSE 推送) 不能设头，所以推送只能用 ? access_token=。

4.5 完整端点地图 (WeFlow 26.5.27，实测于 2026-06-20)

端点来源：从 app.asar 抽取的路由字面量 + 本机逐个活探。比 fork 文档多出一整套朋友圈(SNS)、群成员、推送端点。所有数据接口都强制 Bearer 鉴权。

端点	方法	状态	说明 / 关键字段
/health、 /api/v1/health	GET	✅ 稳定	{"status": "ok"}，无需 token
/api/v1/sessions	GET	✅ 稳定	会话列表：username / displayName / type / sessionType / lastTimestamp / unreadCount
/api/v1/contacts	GET	✅ 稳定	联系人：username / displayName / nickname / type
/api/v1/messages? talker=	GET	⚠️ 通但不稳	消息；最新消息不带 start/end 并用 lastTimestamp 自检，历史区间显式 start/end，结果时有时无 (见下)
/api/v1/sessions/{id} /messages	GET	✅ 同源	/messages?talker= 的路径式等价写法 (空 id→400)
/api/v1/media/{path}	GET	✅ 端点确认	取已导出媒体；带目录穿越防护 (路径越界→403)。受 /messages 不稳定拖累没稳定捞到样本去取文件
/api/v1/group-members? talker=<群id>	GET	✅ 实测可用	群成员：success / chatroomId / count / fromCache / updatedAt / members[]；成员字段含 wxid / displayName / nickname / remark / alias / groupNickname / avatarUrl / isOwner / isFriend / messageCount
/api/v1/sns/timeline? limit=	GET	✅ 实测可用	朋友圈时间线：{success, count, timeline[]}；条目字段 tid / id / username / nickname / createTime / contentDesc / type / media / likes / comments / rawXml / location

端点	方法	状态	说明 / 关键字段
<code>/api/v1/sns/post/{id}</code>	DELETE	写操作 (GET→405)	删除一条朋友圈
<code>/api/v1/sns/usernames</code>	GET	✅ 实测可用	<code>{success,usernames[]}</code> 有朋友圈的用户名列表
<code>/api/v1/sns/export</code>	POST	写操作	触发朋友圈导出
<code>/api/v1/sns/export/stats</code>	GET	✅ 实测可用	<code>{success,data:{totalPosts,totalFriends,myPosts}}</code>
<code>/api/v1/sns/media/proxy</code>	GET	未测 (需参数)	朋友圈媒体代理
<code>/api/v1/sns/block-delete/status</code>	GET	✅ 实测可用	<code>{success,installed:bool}</code> 防朋友圈删除钩子状态
<code>/api/v1/sns/block-delete/install</code>	POST	写操作 (GET→405)	安装防删钩子
<code>/api/v1/sns/block-delete/uninstall</code>	POST	写操作	卸载防删钩子
<code>/api/v1/push/messages?access_token=</code>	GET (SSE)	✅ 实测可用 (开关后)	实时消息推送 SSE 长连接 。事件名 <code>message.new</code> / <code>message.revoke</code> ；私聊字段 <code>rawid</code> / <code>avatarUrl</code> / <code>sourceName</code> / <code>content</code> / <code>timestamp</code> (秒级)，群聊多 <code>groupName</code> ；另有 <code>event</code> / <code>sessionId</code> / <code>sessionType</code> 。有 <code>event: ready</code> 握手、25s 心跳： <code>ping</code> 、断线 <code>Last-Event-ID</code> 回放。可在设置里按会话过滤 (只推/屏蔽指定会话)。详见下方「403 坐实」

注：`/version`、`/info`、`/api/book`、`/api/v3`、`/api/v4` 这些字符串虽在包内出现，但**不是本服务的 HTTP 路由** (实测 404)，请勿调用。

🔗 `push/messages` 的 403 已坐实 = 配置问题，非版本、非调用错误。源码 `handleMessagePushStream` 第一行就是 `if(configService.get('messagePushEnabled')!==true){ sendError(403,'Message push is disabled')}`。实测 Bearer 头 / `?access_token=` / 加 `Accept: text/event-stream` 三种姿势都返回同一个 `403 {"error":"Message push is disabled"}`，与源码逐字一致。解法：到 WeFlow 设置 → API 服务里把「主动推送」开关也打开 (它与「HTTP API 服务」是两个独立开关)。

已实体验证 (2026-06-20)：开启「主动推送」后再连同一地址，返回从 403 变为 `HTTP 200 / Content-Type: text/event-stream`，首段 `event: ready + data: {"success":true,"stream":".../api/v1/push/messages"}` —— 与源码完全一致。用浏览器 `EventSource` 连 `http://127.0.0.1:5031/api/v1/push/messages?access_token=<token>` 即可持续收流。

⚠️ 已知问题 (`/messages` 不稳定)：WeFlow 是“实时读库、无中间解密库”的架构，消息索引疑似惰性/有竞态——相同调用可能这次有数据、下次为 0 (实测同一会话在 50 条 / 0 条之间跳)。建议：① 取最新消息时不带日期，直接 `limit=100`，取返回数组前 N 条，并用 `sessions.lastTimestamp` 对齐自检；② 取历史区间/批量回溯时显式给

start / end ; ③ 返回 0 或失败时**重试几次**；④ 批量分析前先用 `probe-weflow.ps1` 确认当前能否取到消息；⑤ 优先依赖 `sessions / contacts / group-members / sns/*` 这些实测稳定的接口。

典型 Agent workflow： `/api/v1/sessions` 找会话并记录 `lastTimestamp` → (群聊可 `/api/v1/group-members` 拿成员画像) → 最新消息用 `/api/v1/messages?talker=...&limit=100`，历史区间用 `/api/v1/messages?talker=...&start=...&end=...&chatlab=1` → 交给模型做摘要 / 分析 / 报告；朋友圈分析走 `/api/v1/sns/timeline` + `/sns/export/stats`。

5. 安全模型与红线 ⚠

WeFlow API (26.5.27) 的安全模型，**读源码后修正如下**（比早前判断要好，但仍需谨慎）：

1. **只绑定 127.0.0.1**——绝不要用端口转发 / 反代 / `0.0.0.0` 把它暴露到局域网或公网。一旦暴露，等于把全部微信记录裸奔。
2. **数据接口强制 token**——除 `/health` 外都要 `Bearer / access_token`，无 token 一律 `401`。这是主要防线。
3. **CORS 实际是受限的**（源码实证，纠正早前“任意网页可读”的说法）：`Access-Control-Allow-Origin` 只回显匹配 `^https?:/(localhost|127.0.0.1)(:\d+)?$` 的源。随机外部网站（如 `https://evil.com`）拿不到该响应头，浏览器会**拦掉**它读取响应；加之没 token 也是 `401`。所以“任意网页静默偷读”的风险被这两层挡住。- 仍建议：**不用时关掉 API 服务**；本机别同时跑会发 `localhost` 源请求的可疑本地程序；token 不要泄露（拿到 token 仍可读全部数据）。

密钥红线（本仓库为何坚持私有 + gitignore）： - `WEFLOW_DB_KEY`（SQLCipher 库密钥）解密你**全部**聊天历史，且**不可轮换**——进过任何 git 历史 = 永久泄密。 - `WEFLOW_TOKEN` 可在 WeFlow 设置里**重新生成**——本仓库建好后建议立刻轮换一次（它曾出现在对话/剪贴板里）。 - 你有过把 TG token 误推进公开仓库的前例，所以这里**密钥只进 .env**，README 里全用 `wxid_xxx` 占位。 - 本仓库远端是 public；提交前按 `docs/privacy_boundary.md` 做公开安全检查。

6. 快速自检

```
# 本机只有 Windows PowerShell 5.1（无 pwsh），用 powershell 运行：
powershell -ExecutionPolicy Bypass -File E:\WeFlowBridge\probe-weflow.ps1
```

脚本从 `.env` 读 base/token，依次探活 `/health`、`/sessions`、`/contacts`、`/group-members`、`/sns/export/stats`、`/messages`，报告服务是否在线、鉴权是否通过、各接口能否取到数据。脚本本身须为 **UTF-8 BOM** 编码，中文才不会乱码。

7. 来源与致谢

- WeFlow 本体：<https://github.com/hicccc77/WeFlow>
- 本文 API 端点依据其 `docs/HTTP-API.md`（经 fork `RuoCJ/WeFlowBack` 取回，官方主仓库未公开该文件）
- ChatLab 格式：<https://github.com/nichuanfang/chatlab-format>

请负责任地使用，仅处理你本人有权访问的数据，遵守相关法律法规。